

Task 1: Understanding Agile Methodology

Provide a brief overview of Agile methodology, including its key principles and practices.

-Agile is an iterative and incremental approach to software development that organizes work into short cycles, favors adaptability over rigid up-front planning, and keeps the people building the product in constant contact with the people who will use it. Rather than trying to define every requirement before a single line of code is written, Agile teams build a small piece of working software, gather feedback on it, and use that feedback to decide what to build next.

The philosophy behind Agile is captured in the Agile Manifesto, published in 2001 by a group of software practitioners who were frustrated with the inefficiencies of heavyweight, document-driven development processes. The Manifesto is built on four core values: Individuals and interactions over processes and tools — people solving problems together produce better outcomes than any process on its own. Working software over comprehensive documentation — a running increment proves progress more convincingly than a document.

Customer collaboration over contract negotiation — requirements are refined together with the customer rather than locked into a fixed contract.

Responding to change over following a rigid plan — the ability to adapt is treated as more valuable than sticking to an original plan.

These values are expanded into twelve supporting principles, which in practice translate into recurring habits: self-organizing, cross-functional teams; frequent delivery of working software, typically every few weeks rather than months; close, even daily, collaboration between developers and business stakeholders; a sustainable pace of work rather than crunch-driven deadlines; technical excellence and good design as an ongoing concern rather than a one-time milestone; and regular reflection, retrospectives, where the team examines how it is working and adjusts.

Explain how Agile addresses the challenges of traditional software development methods.

-Traditional, "waterfall" software development consists of sequential steps: requirements gathering, design, implementation, testing and deployment; each of these steps is expected to be finished before the next one starts. It seemed to make sense in construction and manufacturing, where it is costly to change it late, but unfortunately it brings up recurring issues when used for software:

- *Late discovery of problems. Testing occurs late in the project, so that misinterpreted requirements and designs only come to light after many months of work, and then it is too late and very costly to correct. Agile's short iterations bring these issues to light in days and weeks and before they are expensive to fix.*

- Unhappy to alter, change. Waterfall considers a new requirement if it is introduced midstream in a project as an interruption of the established plan and may need to be the subject of a formal change control process. Agile, however, has a different view on requirements changing and change being part of the process, in the form of continuous backlog reprioritization.
- Delayed value delivery. Usually, waterfall clients do not get anything useful until the conclusion of the project. With Agile, value (and feedback) begins to flow early and often since they are getting a working increment at the end of each sprint (or continuously, in Kanban).

Weak stakeholder involvement. Waterfall tends to collect requirements a single time at the beginning, and the customer is treated like a bystander until delivery. Agile ensures that the product owner/customer is constantly involved with the project and significantly lowers the chance of creating a product that no one wants.

Limited understanding of progress. Progress is reported in terms of document completion; a waterfall project can look on track for months and then falter without a word of progress being made. Real gains or lack thereof is made visible through frequent demonstrations of working increments within Agile.

Describe at least two popular Agile frameworks (e.g., Scrum, Kanban) and their core components.

*-**Scrum** is the most popular Agile framework. It breaks work into fixed length iterations known as sprints typically two to four weeks in length and establishes a small number of roles, artifacts, and events that structure the work of a team and how they plan, build, and reflect on their work.*

Scrum has three roles: Product Owner, who is the owner and prioritizer of the Product Backlog and represents the voice of the customer; Scrum Master, who is the facilitator of the Scrum process, removes impediments, and protects the Scrum team from disruptions; and the Development Team, the cross-functional team of people who design, build, and test the product increment.

It has a few core artifacts, including the product backlog, which is the comprehensive list of everything that can be built, prioritized by the team; the sprint backlog, which is the subset of items that the team has agreed to build in the sprint; and the increment, which is the working, potentially shippable piece of product that the team produces in the sprint.

It is organized around its events: the sprint planning (to determine what will be built during the sprint), the daily stand-up (to synchronize progress and blockers with stakeholders), the sprint review (to demonstrate the completed increment to stakeholders), and the sprint retrospective (for the team to review how they worked together and to improve things for the next sprint). The Scrum board is a typical method for tracking progress, note that unlike a Kanban board the backlog remains as an artifact off the board; the board only shows the work currently being done in the sprint.

*The **Kanban** is a simpler version of Agile, as it is different from Scrum, in that it does not have a fixed length of iteration and does not have a set of roles. It emphasizes visualizing work and making sure that the amount of work that is flowing up in the system matches the real capacity of the team, so that there is no batching of work (work in progress).*

At the core of a Kanban board are its columns, which are used to represent the different stages of the workflow. These are typically "To Do," "In Progress," and "Done," although teams may have more detailed stages, like "In Review" or "Ready for Deployment. The key mechanism in Kanban is the Work In Progress (WIP) limit: there are limits on the number of items allowed in each column if more items come into the column, they need to get the work out of the column before they can put new work in. This means that when there is a limit, the team will notice a bottleneck and be forced to complete work in a column before adding new work to the column.

Kanban is more flexible and lightweight than Scrum, and is well suited to teams that are regularly receiving work from others that is not necessarily planned in advance, such as support teams or operation teams. The Kanban teams monitor the flow metrics like cycle time (the time taken from start to finish), lead time (how much time the item spends waiting from request to completion), throughput (how many items can be completed in a given unit of time).

They are both based on the same Agile values – iterative delivery, continuous improvement, and close collaboration with stakeholders, but they organize the flow of work in different ways. Scrum is structured by time-boxing commitments and roles, it fits well with teams that can benefit from predictable planning cycles. Kanban is structured by continuous flow and WIP limits, it fits well with teams whose work comes to them unpredictably. In the real world there are many teams that mix the two together and use a version of Scrum that is referred to as Scrumban, which have the same sprint cadence as Scrum, but incorporate the pull-based flow and WIP limits of Kanban.

Task 2: Integrating Agile and DevOps

Explain how Agile and DevOps can be integrated into a unified software development and delivery process.

-The issues that are solved by Agile and DevOps are neighbouring and complementary — just the kinds of things that make them go together. Agile controls what's built and what the sequence of building is: it's about getting it done in smaller, testable pieces, and keeping priorities on track from user feedback and having a backlog and sprint cycle. DevOps is how things are built and get to the user: automation, shared tools and shared ownership between developers and ops, code can rapidly move from dev workstation to production quickly, safely, and repeatably.

Each incremental delivery of the product that is produced is an integration. Instead of the code going into a simple demo process followed weeks later by another manual release process, the code created during the sprint is automatically integrated into a Continuous Integration /

Continuous Delivery (CI/CD) pipeline, where the code is built and tested and also released into production. It is understood by examining this cultural part of this integration under the light of the CALMS framework, which is sometimes referred to as DevOps on top of an already Agile team, where the principles of Agile (collaboration and iteration) are extended with operations.

In reality, this fusion is made possible by a few concrete practices:

Infrastructure as Code and automated environment provisioning environments for a sprint's work were no longer requested in an operations queue; instead they were available on demand. The automated tests are directly in the pipeline, taking Agile's "definition of done" beyond a working demo to passing automated tests.

Shared backlogs containing operational and reliability work (monitoring, scaling, security patching) and feature work so that operational concerns are prioritized in the same manner as user facing features.

Cross-functional teams extending Scrum's cross functional team from development to people with operations and infrastructure skills.

Use feature flags and incremental rollout to enable a team to merge and deploy code continuously without releasing it to users. Develop feature flags and incremental deployments to allow a team to merge and deploy code continuously, but not release it to users.

Discuss the benefits of combining Agile and DevOps for a software development team.

-Greater speed and reliability. There's no more manual handoffs and approval delays between a completed sprint and a live release, as automated pipelines can streamline the process significantly, reducing time-to-market.

Earlier and continuous quality feedback. By implementing Agile's regular testing process and CI/CD's automated build-and-test process, teams can identify defects before they get to the point of being released, making it much more cost-effective.

Greater working together amongst roles. DevOps extends Agile's focus on collaboration and close contact with operations team to the entire delivery chain, rather than just the development team.

Smaller, lower-risk changes. Small increments and DevOps can deploy without any integration - releases are smaller, less risky, frequent and easier to roll back if something goes wrong compared to large releases with high risk of failure.

Close collaboration between production and the backlog. The next sprint's backlog is fed with monitoring and live system telemetry data, which is one of the DevOps practices, and therefore, not only features are planned, but real user behaviour is also a part of the next features built.

Increased morale and decreased burnout. Early detection of issues and the ability to automate repetitive release activities helps to minimize late night firefighting that is typical with infrequent high-stakes releases, which aligns with Agile's principle of a sustainable pace.

Improved alignment of objectives with business objectives. Reduced and more predictable release cycles allow the business to more easily schedule releases, marketing and communicating with customers based on real sales delivery capability, not on best guesses.

Provide an example of how Agile sprints can align with DevOps CI/CD pipelines to enhance software delivery.

-Greater speed and reliability. There's no more manual handoffs and approval delays between a completed sprint and a live release, as automated pipelines can streamline the process significantly, reducing time-to-market.

Earlier and continuous quality feedback. By implementing Agile's regular testing process and CI/CD's automated build-and-test process, teams can identify defects before they get to the point of being released, making it much more cost-effective.

Greater working together amongst roles. DevOps extends Agile's focus on collaboration and close contact with the operations team to the entire delivery chain, rather than just the development team.

Smaller, lower-risk changes. Small increments and DevOps can deploy without any integration - releases are smaller, less risky, frequent and easier to roll back if something goes wrong compared to large releases with high risk of failure.

Close collaboration between production and the backlog. The next sprint's backlog is fed with monitoring and live system telemetry data, which is one of the DevOps practices, and therefore, not only features are planned, but real user behaviour is also a part of the next features built.

Increased morale and decreased burnout. Early detection of issues and the ability to automate repetitive release activities helps to minimize late night firefighting that is typical with infrequent high-stakes releases, which aligns with Agile's principle of a sustainable pace.

Improved alignment of objectives with business objectives. Reduced and more predictable release cycles allow the business to more easily schedule releases, marketing and communicating with customers based on real sales delivery capability, not on best guesses.